



Solving Hard Mizar Problems with Instantiation and Strategy Invention

Jan Jakubův^{1,2(✉)} , Mikoláš Janota¹ , and Josef Urban¹

¹ Czech Technical University in Prague, Prague, Czechia
jakubuv@gmail.com, Mikolas.Janota@cvut.cz, josef.urban@gmail.com

² University of Innsbruck, Innsbruck, Austria

Abstract. In this work, we prove over 3000 previously ATP-unproved Mizar/MPTP problems by using several ATP and AI methods, raising the number of ATP-solved Mizar problems from 75% to above 80%. First, we start to experiment with the cvc5 SMT solver which uses several instantiation-based heuristics that differ from the superposition-based systems, that were previously applied to Mizar, and add many new solutions. Then we use automated strategy invention to develop cvc5 strategies that largely improve cvc5's performance on the hard problems. In particular, the best invented strategy solves over 14% more problems than the best previously available cvc5 strategy. We also show that different classification methods have a high impact on such instantiation-based methods, again producing many new solutions. In total, the methods solve 3,021 (21.3%) of the 14,163 previously unsolved hard Mizar problems. This is a new milestone over the Mizar large-theory benchmark and a large strengthening of the hammer methods for Mizar.

1 Introduction: Mizar, ATPs, Hammers

The Mizar Mathematical Library (MML) [1] is one of the earliest large libraries of formal mathematics, containing a wide selection of lemmas and theorems from various areas of mathematics. The MML and the Mizar system [2, 19, 35] has been used as a source of automated theorem proving (ATP) [46] problems for over 25 years, starting with the export of several Mizar articles done by the ILF system [10, 11]. Since 2003, the MPTP system [52, 53] has been used to export the MML in the DFG [20] and later TPTP [51] formats. In the earliest (2003) ATP experiments over the whole library, state-of-the-art ATPs could prove about 40% of these problems when their premises were limited to those used in the human-written Mizar proofs (the so called *bushy*¹, i.e., easier, mode).

Since 2013, a fixed version of the MML (1147) and MPTP consisting of 57880 problems has been used as a large benchmark for ATPs and related hammer [7] (large-theory) methods over Mizar [9, 21, 28, 41, 42, 50]. When using many ATP and premise-selection methods, 56.2% of the problems could be proved in [30].

¹ <https://tptp.org/MPTPChallenge>.

This was recently raised to 75.5% [24], mainly by using the learning-guided E [48] (ENIGMA [17, 27]) and Vampire [33] (Deepire [49]) systems.

Both E and Vampire are mainly saturation-style superposition systems. In the recent years, instantiation-based systems and satisfiability modulo theories (SMT) solvers such as cvc5 [3], iProver [32] and Z3 [12] are however becoming competitive even for problems that do not contain explicit theories in the SMT sense [6, 13, 18]. The problems that they solve are often complementary to those solved by the superposition-based systems.

1.1 Contributions

In this work, we use instantiation-based methods (Sect. 2) to solve automatically as many hard Mizar problems as possible. Our main result is that the set of ATP-provable MPTP problems has been increased by over 3000, from 75.5% to 80.7%. All these problems are proved by the cvc5 system which we improve in several ways. First, we use the Grackle system [22] (Sect. 3) to automatically invent stronger strategies for MPTP (Sect. 4.3). Our best strategy outperforms the previously best cvc5 strategy by 14% and our best 7-strategy portfolio solves 8.8% more problems than the corresponding CASC portfolio (Sect. 4.4). We also combine strategy development with alternative clausification methods. This turns out to have a surprisingly high impact on the instantiation-based system, contributing many new solutions (Sect. 4.5). Finally, we obtain further solutions by modifying the problems with premise selection (Sect. 4.6). Ultimately, these methods double the number of the previously ATP-unproved Mizar problems solved by cvc5 from 1,534 to 3,021. In this context, we consider a problem proved if there is at least one system that can solve it. We show that the methods extend to previously unseen Mizar problems coming from newly added articulated in a new version of MML (Sect. 4.7). We analyze the invented strategies (Sect. 5) and discuss several hard Mizar problems proved by them (Sect. 6).

2 Instantiation-Based Methods

In contrast to saturation-style superposition systems, SMT solvers, namely cvc5, tackle quantifiers by *instantiations*, which can be seen as a direct application of the Herbrand's theorem. A subformula $(\forall x_1 \dots x_n \phi)$ produces lemmas of the form $(\forall x_1 \dots x_n \phi) \rightarrow \phi[x_1/t_1, \dots, x_n/t_n]$, with ϕ quantifier-free and t_i ground terms. For example, $\forall x R(f(x), c)$ may be instantiated as $(\forall x R(f(x), c)) \rightarrow R(f(c), c)$. Existential quantifiers are removed by Skolemization.

This approach consists of a loop alternating between a *ground solver* and an *instantiation module* (Fig. 1), where the ground solver perceives quantified formulas as opaque propositions. After identifying a model for the ground part, control shifts to the instantiation module. This module generates new instances of the quantified sub-formulas that are currently meant to hold, strengthening the grounded part of the formula. The process stops if the ground part becomes

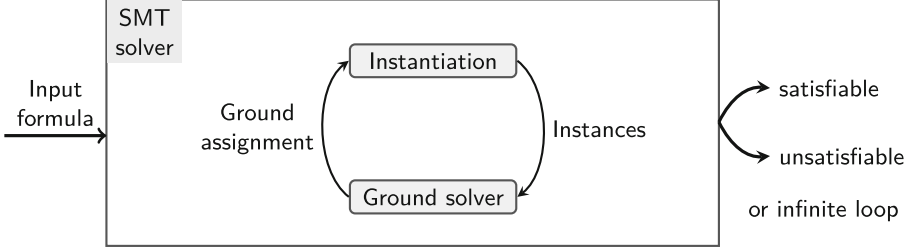


Fig. 1. Schematic of an SMT solver with quantifier instantiation.

unsatisfiable, if ever (model-based quantifier instantiation can also lead to satisfiable answers [16]).

The *cvc5* solver implements several instantiation methods. For decidable fragments, dedicated approaches exist, such as bit-vectors or linear arithmetic [5, 15, 38, 44]. Some of those can be seen as syntactic-driven approaches, *e-matching* [14, 36] or syntax-guided instantiation [39]. Other methods are semantic-driven such as *model-based* [16] or *conflict-based* [45]. A straightforward, but complete, approach for FOL is *enumerative instantiation* [29, 43], which exhaustively goes through all ground terms in the ground part. Instantiation itself can also be guided by ML methods [47].

3 Grackle: Targeted Strategy Invention for Cvc5

Grackle [22] is a system for the automated invention of a portfolio of solver strategies targeted to selected benchmark problems. A user provides a set of benchmark problems and Grackle can automatically discover a set of diverse solver strategies that maximize the number of solved benchmark problems. Grackle supports the invention of good-performing strategies for several solvers, including ATP solvers E [48], Vampire [33], Lash [8], and an SMT solver Bitwuzla [37]. Support for additional solvers can be easily added by providing a parametrization of the solver strategy space, and by implementing a simple wrapper to launch the solver. In this paper, we extend Grackle to support an SMT solver *cvc5* [3], and we evaluate its capabilities on a first-order translation of Mizar problems.

Grackle is a successor of BliStr [54], with which Grackle shares the core of the strategy invention algorithm. Grackle, however, generalizes the algorithm for an arbitrary solver. BliStr/Grackle starts with user-provided solver strategies and interleaves a *strategy evaluation* with a *strategy invention* phase. During the strategy evaluation phase, all available strategies are evaluated on all benchmark problems, typically with some higher resource limit T . This evaluation partitions the benchmark problems by individual strategy performance, giving us, for each strategy S , the set of problems P_S where S performs best. The best strategy S is then *specialized* on problems P_S in the follow-up strategy invention phase in order to search for a strategy S' with an increased performance on P_S . This is

achieved by launching an external parameter tuning software, like ParamILS [23] or SMAC3 [34], on problems P_S with the strategy S as the initial starting point. Moreover, a lower resource limit t than in the evaluation phase (T) is imposed on the solver during the tuning in order to guide the tuner towards an improved performance on P_S . The core idea, verified in previous research [22, 25, 26, 54], is that improved performance on P_S will bring about an improvement on other not-yet-solved problems as well. A new evaluation phase then proceeds with the extended portfolio. Grackle has been extensively described [22] and we refer the reader therein for a detailed exposition.

To use `cvc5` with Grackle requires providing a parametrization of the `cvc5` strategy space. A strategy for `cvc5` is specified as command line options and their values. While `cvc5` supports more than 400 different options, we select all options with non-numeric values relevant to problems in the theory of uninterpreted functions (UF) with quantifiers. This choice is guided by our indented application on the Mizar benchmark problems which are expressed in the UF theory with a large number of quantified formulae. The `cvc5` solver divides its options between *regular* and *expert*. Hence we construct two parametrizations of `cvc5` strategy space, one smaller with the regular options only, and the second one with both regular and expert options. The regular parametrization has 98 parameters and the strategy space covers about 10^{35} different strategies, while the full parametrization has 168 parameters and the space size is about 10^{58} . As an exception, one of the expert options, namely `--cbqi-vo-exp`, was used also in the regular strategy space, to accommodate all the options from the CASC strategies in both spaces. We automatically extract all the options and their values from `cvc5`'s source files [decision_options.toml](#), [prop_options.toml](#), [quantifiers_options.toml](#), [smt_options.toml](#), and [uf_options.toml](#).²

Grackle additionally allows to express dependencies among options and thus to describe options that are effective only under specific settings of another option. We automatically construct some dependencies from common prefixes of option names, for example, the option `--cbqi-mode` is applicable only when the option `--cbqi` is turned on. While many of the dependencies might be left unspecified, and while many of the options might be unrelated to our benchmark problems, we leave this problem to Grackle and to the underlying parameter tuner to deal with. In this way, we also test Grackle's abilities to deal with redundancies in the strategy space. The `cvc5` strategy space for Grackle can be found in the Grackle repository.³

4 Experiments

4.1 Dataset

Our goal is to prove as many of the remaining ATP-unproved MPTP problems as possible. Of the 57,880 problems, 43,717 have been proved⁴ in total in the previ-

² <https://github.com/cvc5/cvc5/tree/cvc5-1.1.1/src/options>.

³ <https://github.com/ai4reason/grackle/tree/v0.2/grackle/trainer/cvc5>.

⁴ https://github.com/ai4reason/ATP_Proofs.

ous experiments [24, 30], thus, 14,163 problems remain to be proved. Our strategy invention methods work by gradually developing strategies that are faster and faster on solvable problems. That is why we extend the set of the 14,163 ATP-unproved problems by another 4,283 hard problems that were proved only in the latest stages of the previous ATP experiments. We will use their versions with heuristically minimized premises (using *subproblem based minimization* [24]) to increase the chances of the ATP systems. We also remove from this set 1,585 problems for which the minimization was not done yet.⁵ This results in a set of 16,861 hard problems on which we develop our strategies. These problems are by default in the FOF format. We denote them min_{fof} below. Later on (Sect. 4.5), we apply different clausifications to them. In Sect. 4.6 we additionally experiment with different premise selections for them.

4.2 Overview of the Experiments

Most of the experiments in this paper are conducted on the set of 16,861 hard Mizar problems described in Sect. 4.1. Section 4.3 focuses solely on describing three Grackle runs performed to develop a robust portfolio of cvc5 strategies specialized for Mizar problems. All strategies are evaluated with a time limit of 30 seconds. Since increasing the time limit can still yield significant improvements, selected strategies are evaluated in Sect. 4.4 with a higher time limit, namely 600 seconds. Sections 4.3 and 4.4 use the same version of problems (min_{fof}) and differ only in the time limit. In Sect. 4.5, we explore different clausification methods, and in Sect. 4.6, we investigate various premise selection methods. This implies that Sects. 4.5 and 4.6 use syntactically different but semantically equivalent versions of Mizar problems. On the other hand, Sect. 4.7 attempts to assess the overfitting of Grackle-invented strategies on a new version of MML, thus the numbers reported therein are not directly comparable with those in previous sections since the problem sets differ.

4.3 Experiments with Grackle Strategy Invention

We evaluate⁶ the Grackle’s ability to invent good-performing strategies for cvc5 on the min_{fof} benchmark. As a baseline, we consider all 16 strategies used in the cvc5’s CASC competition script (see Table 5). We evaluate these 16 strategies with a 30-second time limit per strategy and problem. The best strategy solves 2,508 problems, while all the strategies together solve 3,460 problems. The two most complementary strategies are used as the initial portfolio for the first Grackle run.

We perform consequently three Grackle runs, each with an overall timeout of 7 days. Grackle terminates when all strategies have been already specialized, or when the time is exhausted. All three runs were terminated by timeout.

⁵ These are the non-theorem Mizar toplevel lemmas, for which the subproblem look-up (and thus also minimization) is more challenging.

⁶ On two AMD EPYC 7513 32-Core processors @ 3680 MHz and with 514 GB RAM.

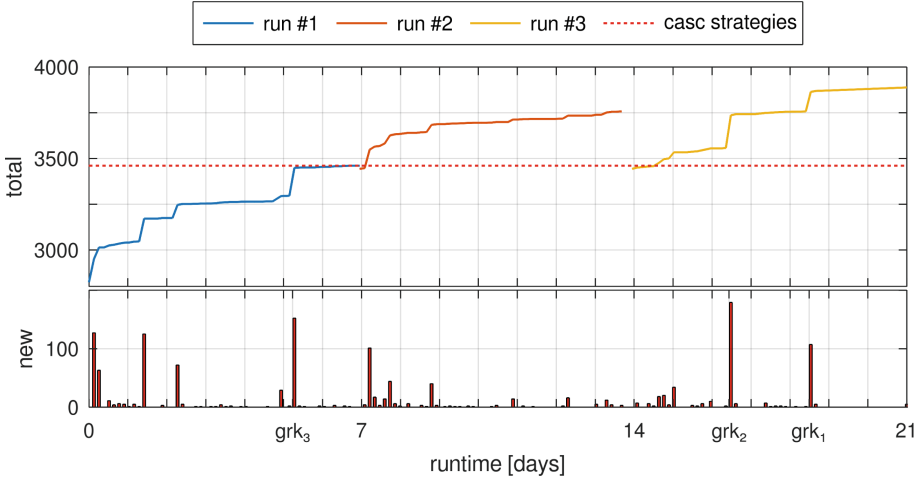


Fig. 2. Time progress of solved problems in Grackle runs.

- (run #1)** The first Grackle run starts with the two most complementary CASC strategies and uses the *regular* strategy space (see Sect. 3). The first run terminated after 7 days with 50 new strategies solving together 3,459 problems with a 30-s time limit per strategy and problem. Out of these problems, 345 are not solved by any of the 16 baseline CASC strategies.
- (run #2)** The second run uses the same regular strategy space as run #1 but it starts from the best 6 strategies found in the first Grackle run. These initial strategies solve 3,425 problems and Grackle invented 45 new strategies solving together 3,696 problems with 485 unsolved by baseline strategies.
- (run #3)** The third run uses the same setup as run #2 but it uses the full strategy space instead of the regular one. Grackle invented 48 new strategies solving together 3,856 problems with 629 unsolved by baseline strategies.

The time progress of the Grackle runs is visualized in Fig. 2. The lower part (*new*) shows the number of new problems solved by the strategy invented at that time, while the upper part (*total*) shows the total number of problems solved. The red dotted line marks the performance of the 16 baseline CASC strategies. The x -axis additionally shows the time of invention of the best three strategies (grk_i).

The figure shows that the expert options added in run #3 helped to develop stronger strategies and to improve the results. Strong strategies are sometimes invented after several days of stagnation. Grackle invented 143 strategies, which together solve 4,113 problems. The best 16 Grackle strategies solve 4,039, which increases the number of 3,460 problems solved by the 16 baseline strategies by 16.7%. The best single strategy solves 2,796, which improves on the best baseline strategy (which solves 2,508) by 11.5%.

Table 1. Grackle strategy invention for cvc5 on Mizar problems (Sect. 4.3).

run	solved				single	strategies			specializations	
	initial	final	new	casc+	best	initial	new	needed	total	failed
#1	2823	3459	+636	+345	2696	2	50	28	56	6
#2	3425	3696	+271	+482	2696	6	45	27	56	11
#3	3425	3856	+431	+629	2796	6	48	29	58	10

Additional data from the experiments are depicted in Table 1. Columns *solved* describe the initial and the final count of solved problems together with the improvement over the initial strategies (*new*), and over the 16 baseline CASC strategies (*casc+*). The column *single best* states the number of problems solved by the best invented strategy. Columns *strategies* describe the *initial* count of strategies, the count of *new* strategies, and how many strategies are needed to cover the *final* number of solved problems (the length of the greedy cover sequence). Columns *specializations* provide the number of attempted specializations (*total*) and *failed* specializations where the output strategy was already known.

4.4 Experiments with Higher Time Limits

The above (Sect. 4.3) strategies were evaluated in a 30 s time limit. To solve more Mizar problems we proceed by evaluating the best strategies with a higher time limit of 600 s. Evaluation of a single strategy with this time limit takes about 20 h. Hence, we evaluate strategies selectively as follows. We start with the strategies evaluated in 30 s and construct their greedy cover. The best 30 s strategy is then evaluated in 600 s. The newly evaluated strategy is then added to the greedy cover and the process is iterated until new problems are being proved. The best Grackle strategy grk_1 solves 3,496 problems, while the best CASC strategy solves 3,059. This is a 14.3% improvement. We ended up with 7 Grackle strategies and with 7 CASC strategies evaluated in 600s solving together 4,653 problems (4,398 by Grackles and 4,043 by CASCs). Together with the strategies evaluated in the lower time limit we solved altogether 5,035 of the benchmark problems at this point.

This experiment with higher time limits also shows an interesting difference in behavior between saturation-based ATP provers and instantiation-based SMT solvers. While SMT solvers seem to benefit significantly from higher time limits, ATP solvers typically benefit much less from it. For example, in the comparable single-strategy setting, cvc5 solves almost 50% more problems when the time limit is increased from 60 to 600 s (see columns *alone* of grk_1 in version *bushy* in Table 2 and Table 4). However, E Prover in the *auto* mode solves only 11% more problems on the same benchmark problems with the same increase of the time limit. A similar relative performance gap is observed across different benchmarks.

4.5 Experiments with Clausification Methods

The Mizar problems are given as TPTP [51] problems in first-order logic (FOF). For `cvc5` we translate them to the SMT2 language [4] in the theory of uninterpreted functions (UF). By default, `cvc5` converts to clausal normal form (CNF) internally but since instantiation-based heuristics seem sensitive to problem reformulation, we also experiment with external clausification. This gives us syntactically different variants of the problems and we can test whether `cvc5` benefits from such alternative ways of clausification.

We use E as the external clausifier and we construct two more problem variants min_{cnf1} and min_{cnf2} . The min_{cnf1} version is produced by using E’s default clausification parameters, while for min_{cnf2} we use a much more aggressive introduction of definitions for frequent subformulas. In particular, this is achieved by the E Prover’s option `--definitional-cnf` with values 24 (default) and 4, respectively. The value indicates how many times a subformula needs to appear for a new definition to be introduced via a new constant. The min_{cnf2} methods compared to min_{cnf1} almost halve the average number of literals in the problems (368.3 vs 668.2) and the average number of symbols drops to 60% (2,512.7 vs 4,124.1).

The best Grackle strategy then solves 3,231 problems in 600 s compared to 3,125 by the best CASC strategy. Both these results use the min_{cnf1} clausification. While the individual performance of the strategies on the externally clausified problems is lower than on the FOF variants, they are indeed highly complementary. Eight Grackle strategies and six CASC strategies evaluated in 600 s increase the number of the solved hard problems from 5,035 to 5,404.

4.6 Experiments on Premise Selection Slices

Based on the success with such problem reformulation, we perform additional experiments, this time with different premise selection methods developed in our prior work [24]. Namely, we evaluate Grackle and baseline strategies on the *bushy* (i.e., not subproblem-minimized) variants of the problems, on the strongest GNN (graph neural network [40]) premise selection slices with the threshold -1 (denoted here `gnn`), and on LightGBM [31] premise selection slices with the threshold 0.1 (`lgbm`). These variants were found complementary in our previous experiments [24]. In a nutshell, the trained GNN puts at all the available premises into a large graph with the edges going between formulas, terms, subterms and symbols, runs several iterations of a *message passing* algorithm in this large graph, and ultimately uses the aggregated information for deciding which of the premises are relevant for the conjecture. The LightGBM instead trains many decision trees on suitable features characterizing the premises to determine their relevance. These methods work quite differently and also sometimes recommend premises that are quite different from the ones used by human formalizers.

We again evaluate our strategies on such problems, first with a lower (60 s) time limit. Table 2 shows a comparative evaluation of the 60 s Grackle and CASC strategies on these slices. In the table, the column *version* displays the

Table 2. Full greedy cover on FOF slices min_{fof} , *bushy*, *gnn*, and *lgbm* with 60 s timeout.

<i>version</i>	<i>strat</i>	<i>addon</i>		<i>total</i>	<i>alone</i>	<i>new</i>
min_{fof}	grk_1	+3034	—	3034	3034	968
<i>gnn</i>	grk_1	+521	+17.17%	3555	1024	336
min_{fof}	grk_3	+486	+13.67%	4041	2828	887
<i>lgbm</i>	grk_1	+264	+6.53%	4305	1260	405
min_{fof}	grk_2	+187	+4.34%	4492	2772	904
<i>bushy</i>	grk_2	+177	+3.94%	4669	963	329
min_{fof}	casc_{10}	+73	+1.56%	4742	2175	598
min_{fof}	casc_{13}	+58	+1.22%	4800	2348	666
<i>gnn</i>	grk_3	+46	+0.96%	4846	930	280
<i>lgbm</i>	grk_2	+30	+0.62%	4876	1183	384
min_{fof}	casc_{14}	+28	+0.57%	4904	2650	828
<i>bushy</i>	casc_{13}	+25	+0.51%	4929	795	239
<i>lgbm</i>	grk_3	+24	+0.49%	4953	1074	313
<i>gnn</i>	grk_2	+23	+0.46%	4976	1000	336
<i>bushy</i>	casc_{14}	+17	+0.34%	4993	913	302
<i>bushy</i>	casc_{10}	+11	+0.22%	5004	609	188
<i>bushy</i>	grk_1	+9	+0.18%	5013	962	319
<i>bushy</i>	grk_3	+8	+0.16%	5021	746	195
<i>gnn</i>	casc_{13}	+7	+0.14%	5028	899	287
<i>lgbm</i>	casc_{10}	+7	+0.14%	5035	895	262
<i>gnn</i>	casc_{14}	+6	+0.12%	5041	954	313
<i>lgbm</i>	casc_{13}	+5	+0.10%	5046	1051	309
<i>lgbm</i>	casc_{14}	+4	+0.08%	5050	1137	363

benchmark version and the column *strat* is the strategy name. The column *addon* describes the addition of the strategy to the portfolio, that is, it lists the number of problems added, and the same in percents. The column *total* lists the cumulative performance of the portfolio up to that line. Finally, the column *alone* shows the individual performance of the strategy and the column *new* shows the number of Mizar problems unproved in our previous research [24,30]. The specification of Grackle-invented strategies (grk_i) can be found in Table 6 while the definitions of CASC strategies is in Table 5. These strategies are further analyzed in Sect. 5.

Based on this, we evaluate the best Grackle strategy grk_1 on all three slices in 600 s. This alone raises the number of solved problems from 5,404 to 6,363. After adding also the 60 s results, we obtain in total 6,469 hard problems solved,

Table 3. Results on MML (left), transfer to new MML (right).

Results on MML				Transfer to new MML			
<i>version</i>	<i>strat</i>	<i>addon</i>	<i>alone</i>	<i>version</i>	<i>strat</i>	<i>addon</i>	<i>alone</i>
$\min_{fof}$	grk ₁	+3496	3496	<i>cnf1</i>	grk ₂	+4861	4861
$\min_{cnf1}$	grk ₂	+738	3231	<i>fof</i>	grk ₁	+433	4541
<i>gnn</i>	grk ₁	+535	1215	<i>cnf1</i>	grk ₃	+164	4495
<i>bushy</i>	grk ₁	+311	1441	<i>fof</i>	casc ₁₃	+78	4406
$\min_{fof}$	grk ₃	+298	3220	<i>fof</i>	grk ₃	+53	4195
<i>lgbm</i>	grk ₁	+233	1512	<i>fof</i>	grk ₂	+39	4418
$\min_{cnf1}$	grk ₃	+161	3223	<i>cnf1</i>	grk ₁	+33	4811
$\min_{cnf1}$	casc ₁₀	+112	3125	<i>cnf1</i>	casc ₁₀	+17	4211
$\min_{fof}$	grk ₂	+90	3146	<i>cnf2</i>	grk ₁	+14	4417
$\min_{cnf2}$	grk ₂	+62	2949	<i>fof</i>	casc ₁₀	+12	3952

of which **3,021** were not proved by ATPs before.⁷ This is our main result. We have proved **21.3%** of the remaining ATP-unproved problems, and increased the total number of *all* ATP-proved Mizar problems to 46,738 (**80.7 %**). About half of the 3,021 problems (1,534) can be solved by the cvc5 CASC strategies. For the remaining half, some of our methods (new strategies, different clausifications or premise slices) are necessary.

The first 10 strategies in the final greedy cover are shown in Table 3 (left). The meaning of the columns is the same as in Table 2, that is, the column *version* displays benchmark version, the column *strat* is the strategy name, the column *addon* describes the addition of the strategy to the portfolio, and *alone* shows the individual performance of the strategy. We can see that the Grackle-invented strategies clearly dominate the greedy cover. While premise selection slices *gnn*, *lgbm*, and *bushy* exhibit low individual performance, they provide many new solutions. This is often due to the alternative proofs proposed by the premise selection methods trained over many previous proofs.

For the sake of completeness, Table 4 additionally presents extended results. The table mixes strategies evaluated with different time limits denoted in the column *timeout*. The meaning of other columns is the same as in Table 2. All Grackle and CASC strategies solve together 6363 Mizar problems.⁸

4.7 Transfer to New MML

To assess the overfitting of the methods we evaluate the best three Grackle and the best three CASC strategies on one more benchmark. We use 13370

⁷ The lists of problems solved by the individual strategies and the strategy definitions are available at <https://github.com/ai4reason/cvc5-grackle-mizar>.

⁸ The strategies not listed in Table 6 (like grk_{169baa}) can be found in our repository (Note 7).

Table 4. Full complete greedy cover on MML problems.

<i>version</i>	<i>timeout</i>	<i>strat</i>	<i>addon</i>		<i>total</i>	<i>alone</i>	<i>new</i>
<i>min_{fof}</i>	600	<i>grk₁</i>	+3496	–	3496	3496	1243
<i>min_{cnf1}</i>	600	<i>grk₂</i>	+738	+21.11%	4234	3231	1192
<i>gnn</i>	600	<i>grk₁</i>	+535	+12.64%	4769	1215	432
<i>bushy</i>	600	<i>grk₁</i>	+311	+6.52%	5080	1441	553
<i>min_{fof}</i>	600	<i>grk₃</i>	+298	+5.87%	5378	3220	1146
<i>lgbm</i>	600	<i>grk₁</i>	+233	+4.33%	5611	1512	541
<i>min_{cnf1}</i>	600	<i>grk₃</i>	+161	+2.87%	5772	3223	1092
<i>min_{cnf1}</i>	600	<i>cas_{c10}</i>	+112	+1.94%	5884	3125	999
<i>min_{fof}</i>	600	<i>grk₂</i>	+90	+1.53%	5974	3146	1131
<i>min_{cnf2}</i>	600	<i>grk₂</i>	+62	+1.04%	6036	2949	1045
<i>min_{fof}</i>	600	<i>grk₅</i>	+49	+0.81%	6085	3086	1063
<i>min_{cnf1}</i>	600	<i>grk₁</i>	+35	+0.58%	6120	3163	1110
<i>min_{cnf2}</i>	600	<i>grk₅</i>	+31	+0.51%	6151	2909	1030
<i>min_{cnf1}</i>	600	<i>grk₅</i>	+27	+0.44%	6178	3113	1099
<i>min_{cnf2}</i>	600	<i>grk₃</i>	+22	+0.36%	6200	2851	934
<i>min_{fof}</i>	600	<i>cas_{c13}</i>	+16	+0.26%	6216	2711	848
<i>min_{cnf2}</i>	600	<i>cas_{c10}</i>	+14	+0.23%	6230	2695	787
<i>min_{fof}</i>	600	<i>cas_{c10}</i>	+13	+0.21%	6243	2575	795
<i>min_{fof}</i>	600	<i>grk_{169baa}</i>	+12	+0.19%	6255	2993	722
<i>min_{cnf1}</i>	600	<i>cas_{c06}</i>	+12	+0.19%	6267	2334	1002
<i>min_{fof}</i>	600	<i>cas_{c09}</i>	+11	+0.18%	6278	1064	150
<i>min_{fof}</i>	600	<i>cas_{c14}</i>	+10	+0.16%	6288	3059	1057
<i>min_{fof}</i>	30	<i>grk_{d73c5e}</i>	+9	+0.14%	6297	2901	986
<i>min_{fof}</i>	600	<i>cas_{c06}</i>	+8	+0.13%	6305	2380	716
<i>min_{fof}</i>	30	<i>grk₃₉₃₇₆₉</i>	+7	+0.11%	6312	2671	803
<i>min_{cnf1}</i>	600	<i>cas_{c13}</i>	+7	+0.11%	6319	2948	977
<i>min_{fof}</i>	600	<i>cas_{c07}</i>	+5	+0.08%	6324	2955	916
<i>min_{fof}</i>	600	<i>cas_{c16}</i>	+5	+0.08%	6329	2976	885
<i>min_{fof}</i>	30	<i>grk_{1fe2d9}</i>	+5	+0.08%	6334	2770	992
<i>min_{cnf2}</i>	600	<i>cas_{c13}</i>	+5	+0.08%	6339	2726	968
<i>min_{fof}</i>	30	<i>grk₀₁₄₅₆₅</i>	+3	+0.05%	6342	2666	849
<i>min_{fof}</i>	30	<i>grk_{043c34}</i>	+3	+0.05%	6345	2544	769
<i>min_{cnf2}</i>	600	<i>cas_{c06}</i>	+3	+0.05%	6348	2090	631
<i>min_{cnf2}</i>	600	<i>grk₁</i>	+3	+0.05%	6351	2817	933
<i>min_{fof}</i>	600	<i>grk₄</i>	+2	+0.03%	6353	3320	859
<i>min_{fof}</i>	30	<i>grk_{166bee}</i>	+2	+0.03%	6355	2671	1163
<i>min_{fof}</i>	30	<i>grk_{04f79f}</i>	+1	+0.02%	6356	2484	725
<i>min_{fof}</i>	30	<i>grk_{0f4750}</i>	+1	+0.02%	6357	2465	723
<i>min_{fof}</i>	30	<i>grk_{1499bd}</i>	+1	+0.02%	6358	2238	641
<i>min_{fof}</i>	30	<i>grk_{1afb4a}</i>	+1	+0.02%	6359	463	41
<i>min_{fof}</i>	30	<i>grk₃₄₀₀₇₅</i>	+1	+0.02%	6360	2556	742
<i>min_{fof}</i>	30	<i>grk_{52ae2f}</i>	+1	+0.02%	6361	2670	800
<i>min_{fof}</i>	30	<i>grk_{7dac18}</i>	+1	+0.02%	6362	173	6
<i>min_{fof}</i>	30	<i>grk_{ba0f42}</i>	+1	+0.02%	6363	1810	509

bushy problems coming from newly added articles in MML version 1382. Table 3 (right) shows the results. The Grackle strategies outperform all CASC strategies, even though the improvement is smaller than on the MML problems they were developed for. Alternative clausification methods again provide a considerable improvement.

5 Analysis of the Invented Strategies

As usual with automated strategy invention, there are many new combinations of parameters that may require deeper analysis to understand the automatically invented behavior. That is why we make them publicly available.⁹ As a baseline and as a starting point for Grackle strategy inventions, we consider 16 strategies used in the cvc5's CASC competition script.¹⁰ The strategies are listed in Table 5. The best Grackle strategies are depicted in Table 6.

Interestingly, the different Grackle-invented strategies focus mainly on changing the behavior of the different components of the quantifier instantiation module of cvc5, cf. Sect. 2. By default cvc5 relies on e-matching [14, 36], which is incomplete, which also means that the solver may quickly give up (return the output **unknown**). The option `--full-saturate-quant`, runs the default mode but if that fails to answer, the solver resorts to the enumerative mode (complete for FOL [43]). This explains why this option is so prevalent in the invented strategies.

In `grk1` and `grk3`, e-matching's behavior is changed by changing the trigger-generation policy. In `grk1` and `grk2`, the option `--cbqi-vo-exp` affects the behavior of the conflict-driven instantiation [45]. The option `--cond-var-split-quant` affects the quantifier splitting policy. The option `--term-db-mode=relevant` enforces a stricter policy on ground term filtering. In general, it seems that the essence of a successful strategy is a combination of enumerative instantiations with an appropriate trigger selection for e-matching. In the next section (Sect. 6) we discuss the influence of such options on the solution of several hard Mizar problems.

6 Interesting Mizar Problems Proved

Since we are focusing on the 25% of the Mizar problems that have not been proved by ATPs so far, the newly solved problems are typically quite involved, with long proofs both in Mizar and in cvc5. 127 of them take more than 100 lines to prove in Mizar, and the average Mizar proof length is 41. This is one page of a proof in a paper like this.

A previously ATP-unproved problem that seems relatively easy for many of the cvc5 strategies is `KURATO_1:6`¹¹ related to the well-known Kuratowski's

⁹ See Note 7.

¹⁰ <https://github.com/cvc5/cvc5/blob/cvc5-1.1.1/contrib/competitions/casc/run-script-cascj11-fof>.

¹¹ http://grid01.ciirc.cvut.cz/~mptp/7.13.01.4.181.1147/html/kurato_1.html#T6.

Table 5. CASC baseline strategies used in the experiments.

name	cvc5 strategy options
casC ₁	--decision=internal --simplification=none --no-inst-no-entail --no-cbqi --full-saturate-quant
casC ₂	--no-e-matching --full-saturate-quant
casC ₃	--no-e-matching --enum-inst-sum --full-saturate-quant
casC ₄	--finite-model-find --uf-ss=no-minimal
casC ₅	--multi-trigger-when-single --full-saturate-quant
casC ₆	--trigger-sel=max --full-saturate-quant
casC ₇	--multi-trigger-when-single --multi-trigger-priority --full-saturate-quant
casC ₈	--multi-trigger-cache --full-saturate-quant
casC ₉	--prenex-quant=none --full-saturate-quant
casC ₁₀	--enum-inst-interleave --decision=internal --full-saturate-quant
casC ₁₁	--relevant-triggers --full-saturate-quant
casC ₁₂	--finite-model-find --e-matching --sort-inference --uf-ss-fair
casC ₁₃	--pre-skolem-quant=on --full-saturate-quant
casC ₁₄	--cbqi-vo-exp --full-saturate-quant
casC ₁₅	--no-cbqi --full-saturate-quant
casC ₁₆	--macros-quant --macros-quant-mode=all --full-saturate-quant

closure-complement problem.¹² The theorem shows that for any set A , its Kurat14Set (i.e., a family of 14 sets created by applying closure and complement operations in a particular way to A) is already closed under complement and closure:

definition

```
let T be non empty TopSpace;
let A be Subset of T;
func Kurat14Set A -> Subset-Family of T equals
{ A, A-, A'-, A'-, A'-, A'-, A'-, A'- } \ /
{ A', A'-, A'-, A'-, A'-, A'-, A'-, A'- };
end;
```

theorem Th6: for T being non empty TopSpace
for A, Q being Subset of T st Q in Kurat14Set A holds
Q' in Kurat14Set A & Q- in Kurat14Set A;

¹² https://en.wikipedia.org/wiki/Kuratowski%27s_closure-complement_problem.

Table 6. Best five strategies invented by Grackle.

name	cvc5 strategy options
grk_1	<code>--cbqi-vo-exp --cond-var-split-quant=agg --full-saturate-quant --relational-triggers</code>
grk_2	<code>--cbqi-vo-exp --full-saturate-quant --miniscope-quant=off --multi-trigger-priority --no-static-learning --relevant-triggers --ieval=off</code>
grk_3	<code>--full-saturate-quant --multi-trigger-priority --multi-trigger-when-single --term-db-mode=relevant</code>
grk_4	<code>--cbqi-vo-exp --cond-var-split-quant=agg --full-saturate-quant --inst-when=last-call</code>
grk_5	<code>--cbqi-all-conflict --full-saturate-quant --inst-when=full-delay --macros-quant --multi-trigger-priority --quant-dsplit=none --quant-dsplit=none --trigger-sel=min-s-all --uf-ss=none</code>

The proof has 131 lines in Mizar, however it indeed seems achievable by instantiation-based methods that gradually enumerate the applications of closure and complement to the skolems and use congruence closure when a more complex term can be shown to be equal to a less complex term. The problem is a combination of equational reasoning and a large case split (14 cases), which is what likely makes it hard for the superposition-based systems. The success may indicate that a full ATP (or AI/TP) solution of the Kuratowski's closure-complement problem may not be too far today, because proposing the **Kurat14Set** and finding automatically a suitable family of 14 distinct sets (to show that 14 is indeed the smallest number) also seems within the reach of today's systems.

The problem **ASYMPT_1:18**¹³ is on the other hand only provable with a single Grackle-invented strategy grk_2 and external clausification, taking 62 s. The problem states that the functions $f(n) = n \bmod 2$ and $g(n) = n + 1 \bmod 2$ are not in the Big O relation (in any direction).

theorem

```
for f,g being Real_Sequence st
  (for n holds f.n = n mod 2) & (for n holds g.n = n+1 mod 2)
  holds ex s,s1 being eventually-nonnegative Real_Sequence
  st s = f & s1 = g & not s in Big_Oh(s1) & not s1 in Big_Oh(s)
```

The Mizar proof has 122 lines and again goes through several case splits related to the mod 2 values. However a lot of knowledge (often equational) about the

¹³ <http://grid01.ciirc.cvut.cz/~mptp/7.13.01.4.181.1147/html/asymp1.html#T18>.

arithmetical expressions, modulo and inequality has to be applied too.¹⁴ The fact that this can be done by an instantiation-based system is quite remarkable, and probably also due to the fact that the terms that arise in the proof are not extremely complicated thanks to the $\{0,1\}$ codomain of the functions involved. The option `--multi-trigger-priority` seems indispensable for solving the problem, showing the importance of the heuristics for handling instantiation triggers. This may be an opportunity for further AI/ML methods learning even finer control of the triggers in such systems.

Finally, theorem `ROBBINS4:3`¹⁵ shows an equivalent condition for ortholattices:

```
for L being non empty OrthoLattStr holds L is Ortholattice iff
  (for a, b, c being Element of L holds
    (a "\/" b) "\/" c = (c' "\/" b')' "\/" a)
& (for a, b being Element of L holds a = a "\/" (a "\/" b))
& for a, b being Element of L holds a = a "\/" (b "\/" b')
```

The problem can only be solved by the Grackle-invented strategy 89fc24 and it takes 137s. The Mizar proof has 145 lines and uses a lot of equational reasoning in lattice theory. It is quite surprising that a proof with so much equality could not be done by the superposition based systems, and that it can be done by `cvc5`. Again, triggers seem important here, together with the `--term-db-mode=relevant` option which further limits the sets of possible quantifier instantiations.

7 Conclusions and Future Work

We have solved **3,021** (21.3%) of the remaining 14,163 hard Mizar problems, raising the percentage of automatically proved Mizar problems from 75.5% to **80.7%**. This was mainly done by automatically inventing suitable instantiation-based strategies for the `cvc5` solver, using our Grackle system. Further improvements were obtained by using alternative clausifications of the problems, and also alternative premise selections. Such problem transformations have a surprisingly large effect on the instantiation-based procedures and are likely to be explored further when creating strong portfolios for such systems.

The invented `cvc5` strategies perform well also on a set of new problems added in a later version of the Mizar library, showing only limited overfitting. Given today's `cvc5`'s good performance on corpora such as Isabelle/Sledgehammer and TPTP, it may be also interesting to repeat our strategy invention experiments for the TPTP problems and for problems exported from various non-Mizar hammer systems. In general, training instantiation-based systems in various ways is an emerging research topic that may bring interesting improvements to some of today's strongest ATP/SMT methods.

¹⁴ Note that the Mizar/MPTP translation translates everything as uninterpreted functions, i.e., there is no reliance on the arithmetical theories implemented in `cvc5`.

¹⁵ http://grid01.ciirc.cvut.cz/~mptp/7.13.01_4.181.1147/html/robbins4.html#T3.

Acknowledgments. Supported by the Czech MEYS under the ERC CZ project no. LL1902 *POSTMAN*, by Amazon Research Awards, by EU ICT-48 2020 project no. 952215 *TAILOR*, by ERC PoC grant no. 101156734 *FormalWeb3*, by CISCO grant no. 2023-322029, and co-funded by the European Union under the project *ROBOPROX* (reg. no. CZ.02.01.01/00/22.008/0004590).

References

1. Bancerek, G., et al.: The role of the Mizar mathematical library for interactive proof development in Mizar. *J. Autom. Reason.* **61**(1–4), 9–32 (2018)
2. Bancerek, G., et al.: Mizar: state-of-the-art and beyond. In: Kerber, M., Carette, J., Kaliszyk, C., Rabe, F., Sorge, V. (eds.) *CICM 2015. LNCS (LNAI)*, vol. 9150, pp. 261–279. Springer, Cham (2015). https://doi.org/10.1007/978-3-319-20615-8_17
3. Barbosa, H., et al.: cvc5: a versatile and industrial-strength SMT solver. In: Fisman, D., Rosu, G. (eds.) *TACAS 2022. LNCS*, vol. 13243, pp. 415–442. Springer, Cham (2022). https://doi.org/10.1007/978-3-030-99524-9_24
4. Barrett, C., Stump, A., Tinelli, C., et al.: The SMT-LIB standard: version 2.0. In: *Proceedings of the 8th International Workshop on Satisfiability Modulo Theories (Edinburgh, UK)*, vol. 13, p. 14 (2010)
5. Bjørner, N., Janota, M.: Playing with quantified satisfaction. In: *20th International Conferences on Logic for Programming, Artificial Intelligence and Reasoning – Short Presentations, LPAR*, vol. 35, pp. 15–27. EasyChair (2015)
6. Blanchette, J.C., Böhme, S., Paulson, L.C.: Extending sledgehammer with SMT solvers. *J. Autom. Reason.* **51**(1), 109–128 (2013)
7. Blanchette, J.C., Kaliszyk, C., Paulson, L.C., Urban, J.: Hammering towards QED. *J. Formalized Reason.* **9**(1), 101–148 (2016)
8. Brown, C.E., Kaliszyk, C.: Lash 1.0 (system description). In: Blanchette, J., Kovács, L., Pattinson, D. (eds.) *IJCAR 2022. LNCS*, vol. 13385, pp. 350–358. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-10769-6_21
9. Chvalovský, K., Korovin, K., Piepenbrock, J., Urban, J.: Guiding an instantiation prover with graph neural networks. In: *LPAR. EPiC Series in Computing*, vol. 94, pp. 112–123. EasyChair (2023)
10. Dahn, I.: Interpretation of a Mizar-like logic in first order logic. In: Caferra, R., Salzer, G. (eds.) *FTP 1998. LNCS (LNAI)*, vol. 1761, pp. 137–151. Springer, Heidelberg (2000). https://doi.org/10.1007/3-540-46508-1_9
11. Dahn, I., Wernhard, C.: First order proof problems extracted from an article in the MIZAR mathematical library. In: Bonacina, M.P., Furbach, U. (eds.) *International Workshop on First-Order Theorem Proving (FTP 1997)*, RISC-Linz Report Series No. 97-50, pp. 58–62. Johannes Kepler Universität, Linz (1997)
12. de Moura, L., Bjørner, N.: Z3: an efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) *TACAS 2008. LNCS*, vol. 4963, pp. 337–340. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-78800-3_24
13. Desharnais, M., Vukmirovic, P., Blanchette, J., Wenzel, M.: Seventeen provers under the hammer. In: *ITP. LIPIcs*, vol. 237, pp. 8:1–8:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2022)
14. Detlefs, D., Nelson, G., Saxe, J.B.: Simplify: a theorem prover for program checking. *J. ACM* **52**(3), 365–473 (2005)
15. Farzan, A., Kincaid, Z.: Strategy synthesis for linear arithmetic games. *Proc. ACM Program. Lang.* **2**(POPL), 61:1–61:30 (2018)

16. Ge, Y., de Moura, L.: Complete instantiation for quantified formulas in satisfiability modulo theories. In: Bouajjani, A., Maler, O. (eds.) CAV 2009. LNCS, vol. 5643, pp. 306–320. Springer, Heidelberg (2009). https://doi.org/10.1007/978-3-642-02658-4_25
17. Goertzel, Z.A., Chvalovský, K., Jakubův, J., Olšák, M., Urban, J.: Fast and slow enigmas and parental guidance. In: Konev, B., Reger, G. (eds.) FroCoS 2021. LNCS (LNAI), vol. 12941, pp. 173–191. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-86205-3_10
18. Goertzel, Z.A., Jakubův, J., Kaliszyk, C., Olšák, M., Piepenbrock, J., Urban, J.: The Isabelle ENIGMA. In: ITP. LIPIcs, vol. 237, pp. 16:1–16:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2022)
19. Grabowski, A., Kornilowicz, A., Naumowicz, A.: Mizar in a nutshell. *J. Formalized Reasoning* **3**(2), 153–245 (2010)
20. Hähnle, R., Kerber, M., Weidenbach, C.: Common syntax of the DFGSchwerpunktprogramm deduction. Technical report TR 10/96, Fakultät für Informatik, Universität Karlsruhe, Karlsruhe (1996)
21. Holden, E.K., Korovin, K.: Graph sequence learning for premise selection. *CoRR*, abs/2303.15642 (2023)
22. Hůla, J., Jakubův, J., Janota, M., Kubej, L.: Targeted configuration of an SMT solver. In: Buzzard, K., Kutsia, T. (eds.) CICM 2022. LNCS, vol. 13467, pp. 256–271. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-16681-5_18
23. Hutter, F., Hoos, H.H., Leyton-Brown, K., Stützle, T.: ParamILS: an automatic algorithm configuration framework. *J. Artif. Intell. Res.* **36**, 267–306 (2009)
24. Jakubův, J., et al.: MizAR 60 for Mizar 50. In: ITP. LIPIcs, vol. 268, pp. 19:1–19:22. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2023)
25. Jakubův, J., Suda, M., Urban, J.: Automated invention of strategies and term orderings for vampire. In: GCAI. EPiC Series in Computing, vol. 50, pp. 121–133. EasyChair (2017)
26. Jakubův, J., Urban, J.: BliStrTune: hierarchical invention of theorem proving strategies. In: Bertot, Y., Vafeiadis, V. (eds.) Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs, CPP 2017, Paris, France, 16–17 January 2017, pp. 43–52. ACM (2017)
27. Jakubův, J., Urban, J.: ENIGMA: efficient learning-based inference guiding machine. In: Geuvers, H., England, M., Hasan, O., Rabe, F., Teschke, O. (eds.) CICM 2017. LNCS (LNAI), vol. 10383, pp. 292–302. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-62075-6_20
28. Jakubův, J., Urban, J.: Hammering Mizar by learning clause guidance. In: Harrison, J., O’Leary, J., Tolmach, A. (eds.) 10th International Conference on Interactive Theorem Proving, ITP 2019, 9–12 September 2019, Portland, OR, USA. LIPIcs, vol. 141, pp. 34:1–34:8. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2019)
29. Janota, M., Barbosa, H., Fontaine, P., Reynolds, A.: Fair and adventurous enumeration of quantifier instantiations. In: Formal Methods in Computer-Aided Design (2021)
30. Kaliszyk, C., Urban, J.: MizAR 40 for Mizar 40. *J. Autom. Reasoning* **55**(3), 245–256 (2015)
31. Ke, G., et al.: LightGBM: a highly efficient gradient boosting decision tree. In: NIPS, pp. 3146–3154 (2017)
32. Korovin, K.: iProver – an instantiation-based theorem prover for first-order logic (system description). In: Armando, A., Baumgartner, P., Dowek, G. (eds.) IJCAR 2008. LNCS (LNAI), vol. 5195, pp. 292–298. Springer, Heidelberg (2008). https://doi.org/10.1007/978-3-540-71070-7_24

33. Kovács, L., Voronkov, A.: First-order theorem proving and VAMPIRE. In: Sharygina, N., Veith, H. (eds.) CAV 2013. LNCS, vol. 8044, pp. 1–35. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-39799-8_1
34. Lindauer, M., et al.: SMAC3: a versatile bayesian optimization package for hyperparameter optimization (2021)
35. Matuszewski, R., Rudnicki, P.: Mizar: the first 30 years. *Mech. Math. Appl.* **4**, 3–24 (2005)
36. Moskal, M., Lopuszanski, J., Kiniry, J.R.: E-matching for fun and profit. In: Krstic, S., Oliveras, A. (eds.) Proceedings of the 5th International Workshop on Satisfiability Modulo Theories, SMT@CAV 2007. Electronic Notes in Theoretical Computer Science, Berlin, Germany, 1–2 July 2007, vol. 198, pp. 19–35. Elsevier (2007)
37. Niemetz, A., Preiner, M.: Bitwuzla at the SMT-COMP 2020. *CoRR*, abs/2006.01621 (2020)
38. Niemetz, A., Preiner, M., Reynolds, A., Barrett, C., Tinelli, C.: Solving quantified bit-vectors using invertibility conditions. In: Chockler, H., Weissenbacher, G. (eds.) CAV 2018. LNCS, vol. 10982, pp. 236–255. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-96142-2_16
39. Niemetz, A., Preiner, M., Reynolds, A., Barrett, C., Tinelli, C.: Syntax-guided quantifier instantiation. In: TACAS 2021. LNCS, vol. 12652, pp. 145–163. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-72013-1_8
40. Olsák, M., Kaliszyk, C., Urban, J.: Property invariant embedding for automated reasoning. In: De Giacomo, G., et al. (eds.) ECAI 2020 - 24th European Conference on Artificial Intelligence. Frontiers in Artificial Intelligence and Applications, vol. 325, pp. 1395–1402. IOS Press (2020)
41. Rawson, M., Reger, G.: A neurally-guided, parallel theorem prover. In: Herzig, A., Popescu, A. (eds.) FroCoS 2019. LNCS (LNAI), vol. 11715, pp. 40–56. Springer, Cham (2019). https://doi.org/10.1007/978-3-030-29007-8_3
42. Rawson, M., Reger, G.: lazyCoP: lazy paramodulation meets neurally guided search. In: Das, A., Negri, S. (eds.) TABLEAUX 2021. LNCS (LNAI), vol. 12842, pp. 187–199. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-86059-2_11
43. Reynolds, A., Barbosa, H., Fontaine, P.: Revisiting enumerative instantiation. In: Beyer, D., Huisman, M. (eds.) TACAS 2018. LNCS, vol. 10806, pp. 112–131. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-89963-3_7
44. Reynolds, A., King, T., Kuncak, V.: Solving quantified linear arithmetic by counterexample-guided instantiation. *Formal Methods Syst. Des.* **51**(3), 500–532 (2017)
45. Reynolds, A., Tinelli, C., de Moura, L.M.: Finding conflicting instances of quantified formulas in SMT. In: Formal Methods in Computer-Aided Design, FMCAD, pp. 195–202. IEEE (2014)
46. Robinson, J.A., Voronkov, (eds.): Handbook of Automated Reasoning (in 2 Volumes). Elsevier and MIT Press (2001)
47. Janota, M., Piepenbrock, J., Piotrowski, B.: Towards learning quantifier instantiation in SMT. In: Meel, K.S., Strichman, O. (eds.) 25th International Conference on Theory and Applications of Satisfiability Testing, SAT 2022, 2–5 August 2022, Haifa, Israel. LIPIcs, vol. 236, pp. 7:1–7:18. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2022)
48. Schulz, S.: System description: E 1.8. In: McMillan, K., Middeldorp, A., Voronkov, A. (eds.) LPAR 2013. LNCS, vol. 8312, pp. 735–743. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-45221-5_49

49. Suda, M.: Improving ENIGMA-style clause selection while learning from history. In: Platzer, A., Sutcliffe, G. (eds.) CADE 2021. LNCS (LNAI), vol. 12699, pp. 543–561. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-79876-5_31
50. Suda, M.: Vampire with a brain is a good ITP hammer. In: Konev, B., Reger, G. (eds.) FroCoS 2021. LNCS (LNAI), vol. 12941, pp. 192–209. Springer, Cham (2021). https://doi.org/10.1007/978-3-030-86205-3_11
51. Sutcliffe, G., Suttner, C., Yemenis, T.: The TPTP problem library. In: Bundy, A. (ed.) CADE 1994. LNCS, vol. 814, pp. 252–266. Springer, Heidelberg (1994). https://doi.org/10.1007/3-540-58156-1_18
52. Urban, J.: MPTP - motivation, implementation, first experiments. *J. Autom. Reasoning* **33**(3–4), 319–339 (2004)
53. Urban, J.: MPTP 0.2: design, implementation, and initial experiments. *J. Autom. Reasoning* **37**(1–2), 21–43 (2006)
54. Urban, J.: BliStr: the blind strategymaker. In: Gottlob, G., Sutcliffe, G., Voronkov, A. (eds.) Global Conference on Artificial Intelligence, GCAI 2015. EPiC Series in Computing, Tbilisi, Georgia, 16–19 October 2015, vol. 36, pp. 312–319. EasyChair (2015)